Tracking4All Documentation

Early Access V1.08

Contents

Tracking4All Documentation....1
Introduction....3
Overview....3
Key Features....3
Getting Started....3
Quick Start Guide....3
Setting up Your Project....3
Core Concepts....4
Architecture Overview....4
Basic Workflow....4
Core Datatypes....6
Framework Guides....6
Runtime Settings....6
Getting and Setting Runtime Settings....7
Adding New Custom Runtime Settings....7
Measuring Landmark Occlusion/Confidence....8
Mirroring and Perspective Flip....8
Smoothing Options....9
Platform Specific Notes....9
Standalone:....9
Mobile:....9
Linux:....9
Experimental Solutions....10
Avatars/Puppeteering....10
Adding a New Avatar....10
Avatar 3D Model Recommendations....11
Creating Avatars at Runtime....11
Avatar Controller Examples....12

© 2024-2025 G8gaming Ltd.

Human Segmentation 13
Segmentation Shader Examples 13
Holistic Hand-Pose (Holistic Lite) 14
Holistic Lite Configuration Options 14
Holistic Lite Limitations 14
Avatars with Face Tracking 15
Vroid Character Pipeline for Face Tracking 16
Advanced Usage 17
Providers 17
Provider Lifecycle 17
Adapter Paradigm and Standardized Adapters 18
IProvider and InterfaceProvider Peculiarities 18
Optimization Tips 19
Support 19
Getting Support 19
License and Acknowledgements 19
License Information 19
Credits and Acknowledgements 19

© 2024-2025 G8gaming Ltd.

Introduction

Overview

Tracking4All is a modular and flexible framework that can be used to power the body tracking features of games/projects. It focuses on modularity and practicality in the context of project prototyping and production. It aims to be backend independent meaning that the features and solutions in the framework should work with a variety of backend tracking/detection approaches varying from Mediapipe to custom solutions. This makes the framework future proof as well as very flexible for use in a variety of cases.

Key Features

- Support for; pose tracking, hand tracking, simultaneous pose and hand tracking, face tracking (coming soon).
- Independent to backend solution (can be powered by solutions ranging from Python server to embedded C++ solutions). Includes pre-baked adapters for most use cases.
- Cross-platform (Windows, Mac, Linux, Android, iOS).
- Aims to be powered by hardware that is accessible to end-users (webcam with no unreasonable setup requirements).
- Modular Features: can optionally add features that extend on the core framework (ex: avatar puppeteering, landmark modifiers, avatar face tracking, etc.) – again independent to the backend data provider.

Getting Started

Quick Start Guide

The fastest way to get started is to open one of the example scenes at 'Tracking4All/Scenes'. Running play will automatically connect to your default webcam and show the solution running.

Either work in a duplication of these examples scenes or use the Tracking Solution prefabs in your scene.

You should NOT develop your project specific assets inside the Tracking4All folder. This makes it difficult to update! During updates it is strongly recommended to replace all Tracking4All assets. It is recommended to make derivative prefabs of the stock ones provided with Tracking4All if you want to customize them and still be able to update easily later.

Setting up Your Project

It is highly recommended to use the project files provided as it includes special player settings and configurations to allow the framework to function and build correctly.

If absolutely required, you can copy the assets into your custom unity project. You may need to adjust player settings and other files to ensure the framework builds correctly.

© 2024-2025 G8gaming Ltd.

Core Concepts

Architecture Overview

The core of Tracking4All can be thought to conceptually consist of 3 layers:

1. Detection Layer: Something needs to provide detection data to Tracking4All (e.g. Landmark data, etc.). Tracking4All doesn't care what the Detection Layer is or how it's powered – however the layer must provide at least the necessary data to power the solutions you want to use. Tracking4All ships with an embedded detection layer (C++) and a simple server-based detection layer (Python).
2. Adapter Layer: The Adapter Layer is responsible for parsing data from the Detection Layer and converting it into data that Tracking4All understands. Users hoping to integrate custom backends should look into this layer.
3. Solutions Layer: The Solutions Layer exposes data that can be read/used in a friendly manner by you and other plug-ins. This layer is easy to use and guaranteed to be Unity thread safe when used properly. Examples of solutions include Puppets and Avatar systems.

Additional non-essential layers:

- Modifier Layer: The Modifier Layer post processes data on the Solutions Layer (e.g. smoothing, transformations, etc.) Modifiers can generically act on as many types of data as the modifiers are implemented to support.

Basic Workflow

1. Solutions are premade for you in the 'Tracking4All/Scenes' folder. It is highly recommended to use the solutions and Tracking Solution prefabs here to begin with.
2. Once you have decided on which solution to use (e.g. Pose). Now decide which solution provider you will use:
    - Just answer the question: do I want detection to run on the target device or remotely? Tracking4All ships with an embedded detection layer (C++) and a basic server-based detection layer (Python).
    - You can choose which detection layer is powering the framework by enabling/disabling the appropriate game object starting with 'MPU' (Mediapipe Unity) or 'MPP' (Mediapipe Python) and then updating the Adapter's provider fields using the unity inspector.
3. You read data from Solutions using Providers. You can check what type of data Solutions can provide by inspecting the Solution class definition. For example, check the Pose Solution:

```
public class PoseSolution : Solution,
    ILandmarkProvider<PoseLandmarks>, INormalizedLandmarkProvider<PoseLandmarks>
```


© 2024-2025 G8gaming Ltd.

You can see that this PoseSolution can provide Landmark and NormalizedLandmark data. Here is a very basic example of how to access data from this PoseSolution:

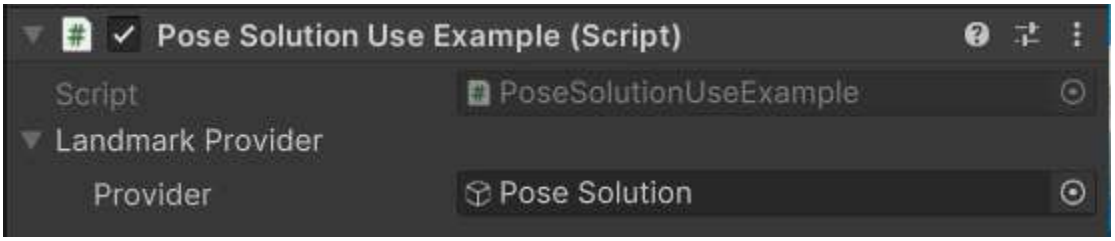
```
public class PoseSolutionUseExample : MonoBehaviour
{
    public LandmarkProvider<PoseLandmarks> landmarkProvider;

    Unity Message | 0 references
    private void OnEnable()
    {
        // Subscribe to landmark provider updates.
        landmarkProvider.OnLandmarksUpdated += LandmarkProvider_OnLandmarksUpdated;
    }
    Unity Message | 0 references
    private void OnDisable()
    {
        // Unsubscribe from landmark provider updates when this gameobject is disabled.
        landmarkProvider.OnLandmarksUpdated -= LandmarkProvider_OnLandmarksUpdated;
    }

    2 references
    private void LandmarkProvider_OnLandmarksUpdated(int group)
    {
        // Get the landmark NOSE from the landmark provider.
        Landmark nose = landmarkProvider.Provider.Get(group, PoseLandmarks.NOSE);

        // Print out the position of the nose.
        print(nose.Position);
    }
}
```

Then create a new gameobject, add the new component, and assign the gameobject which contains the PoseSolution in the inspector like so:



After pressing play, the position of the nose will be output in the console.

© 2024-2025 G8gaming Ltd.

Core Datatypes

Landmark: contains 3D coordinates from the detection backend, presence (whether the landmark is present within the scene bounds), visibility (whether the landmark is occluded).

- NOTE: Only the pose solution currently support presence and visibility values.

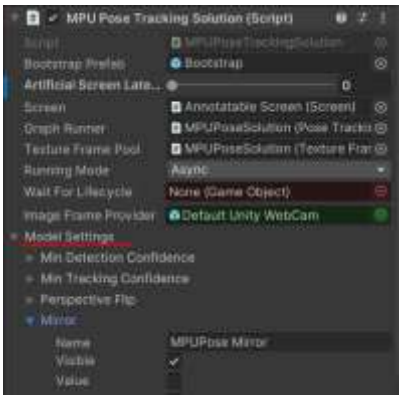
NormalizedLandmark: coordinates of a landmark in camera screen space in a [0,1] range. XY coordinates [0,0] is the bottom left of the camera view and [1,1] is the top right of the camera view. The Z-component is the approximate depth of the landmark.

ImageFrame: essentially wraps a texture + some additional characteristics about that texture.

Framework Guides

Runtime Settings

For your convenience Tracking4All includes a basic settings system for centralized control of parameter driving solutions or for settings relevant to your project. For example, the Mirror option can be found under Model Settings of the MPUPoseTrackingSolution:



Each setting has three parameters:

- Name: the unique name identifying the setting, shows up in the menu with this label.
- Visible: whether or not the setting shows up in the Tracking4All menu (it will still be programmatically accessible regardless).
- Value: the default value for this setting.

© 2024-2025 G8gaming Ltd.

Getting and Setting Runtime Settings

You can get and set the value of any setting visible in the menu by fetching it from the SettingsManager by name. For example:



Get the first setting by: SettingsManager.GetFloat("MPUPose Min Tracking Confidence", 0);

- Will retrieve the actual value or 0 if the setting does not exist.

Set the second setting by: SettingsManager.SetBool("MPUPose Perspective Flip", true);

- Will set the value to true, logs a warning message if the setting does not exist.

Check if a setting exists using: SettingsManager.HasSettings("settingName");

You can subscribe to SettingsManager.OnSettingChanged to receive a callback every time a setting is changed and applied.

You **must** apply any changes to settings using SettingsManager.Apply();

Normally most settings can be accessed during the Start methods execution in Unity.

- NOTE: some settings can be added asynchronously (not in sync with the unity main thread – for example the webcam index), therefore it is generally recommended to wait for the availability of settings before changing them. See ChangeGameSettingExample.cs in the framework Examples folder for an approach to handle this.

Adding New Custom Runtime Settings

It is very easy to add new runtime settings which can be modified in the menu. For example, add a new float setting to the menu with this class in the scene:

```
Unity Script | 0 references
public class RandomSettingAdditionExample : MonoBehaviour
{
    public FloatSetting setting;

    Unity Message | 0 references
    private void OnEnable()
    {
        SettingsManager.Instance.AddSetting(setting);
    }
    Unity Message | 0 references
    private void OnDisable()
    {
        SettingsManager.Instance.RemoveSetting(setting);
    }
}
```

- You can read and write to the setting by getting setting.Value and using seting.Set().
- NOTE: you must still call SettingsManager.Apply() for any changes to take effect.

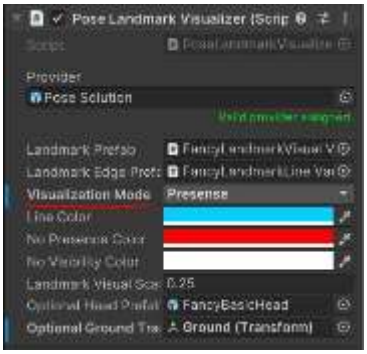
© 2024-2025 G8gaming Ltd.

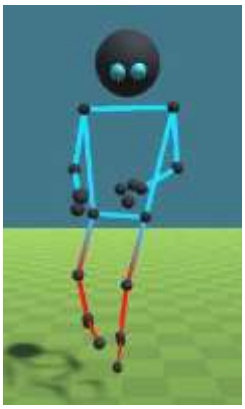
Measuring Landmark Occlusion/Confidence

It can be important to know whether landmarks are being occluded and how to test for reliability of the environment/user setup. Tracking4All exposes 2 values to measure this:

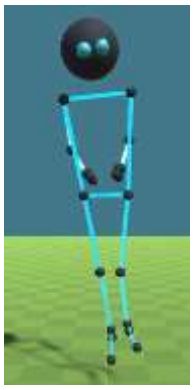
- Visibility: Where 1.0 is completely visible, 0.0 is not visible. This includes if the landmark is hidden by another body part/object or out of the bounds of the camera view.
- Presence: Where 1.0 is completely present, 0.0 is not present. This measures exclusively if a landmark is in the camera view bounds or not.

You can visualize these two parameters using the PoseLandmarkVisualizer. Currently only the pose solution supports these two values.





Lower Torso is out of camera view measured using Presence (colored red).



Hands are hidden behind back, measured using Visibility (coloured white).

Mirroring and Perspective Flip

Tracking4All supports perspective flipping as well as mirroring of pose. The mirror option for most solutions can be accessed through the Menu at runtime or by settings the default value at the source of the setting.

Perspective Flipping: When you want to view the "other side" of the pose. View equivalent to rotating the game camera 180 degrees to see the other side of the pose.

Mirroring: When you want to flip the right and left of the pose so that your right hand controls the left of the avatar (i.e. like a real life mirror).

IMPORTANT: all front facing API calls are relative to the player user. So if you ask the API "what is the position of the right hand" it will return the position of the right hand of the actual user regardless of mirror or perspective flip options (if not that would be a bug).

- When using mirroring with Hand Solutions, it is recommended to read data from the PairHandSolution and PairHandPuppet which respect mirroring settings for handedness.

Another useful way of thinking about mirroring is whether the user and avatar's directionality are logically synchronized.

- Mirror enabled: the user and avatar right/left are opposite (just like with a real-life mirror).
- Mirror disabled: the user and avatar right/left are synced.

© 2024-2025 G8gaming Ltd.

Smoothing Options

Tracking4All currently supports 2 smoothing options for data through the modifier system (see the various ModifierStacks in the Unity inspector).

EuroSmooth: Excels at reducing noise but can also filter out rapid small changes/oscillating movements. Higher frequency values (around 500 will smooth data more, lower will be more responsive but also less smooth).

SimpleSmooth: General multi-purpose smooth higher smooth speed will be more responsive but at the cost of increased noise. Prefer the EuroSmooth if you want 'high level smooth movements'.

Platform Specific Notes

For customizing the preferred inference mode for embedded mediapipe see the AppSettings file located in /Third Party/MediaPipeUnity/MediaPipeUnity/.

Standalone:

- Mediapipe only supports CPU inference on Windows and Mac.

Mobile:

- Use GPU inference on mobile for performance (this is automatically used in most cases).
- The Perspective Flip and Mirror settings will be useful for front facing cameras.
- Enabling segmentation is not recommended on mobile due to its heavy performance requirements (but if you're stubborn: set MPUPoseTrackingSolution running mode to Async and use a mobile optimized shader).
    - NOTE: you can still use the SegmentationController example to still have an avatar follow the user in the frame.
- See Optimization Tips.

Linux:

- Mediapipe supports both GPU and CPU inference on Linux. Prefer Vulkan graphics API.
  Note: while GPU performance can be a little better, you are often bound by webcam frame rate, so CPU inference is generally still recommended.
- You may need to install some packages for Linux from the Unity Registry and cleanup third-party assets or else you may get compile errors (ex: Unity UI).

© 2024-2025 G8gaming Ltd.

Experimental Solutions

Avatars/Puppeteering

Puppeteering arbitrary humanoid avatars is supported however is a little bit rough around the edges currently. This will improve over time. Tracking4All generates full transform data given a pose data provider by driving a Puppet. Avatars then retrieve the transform data from a puppet to drive the motion of the avatar.

Features:

- Supports arbitrary tracked rotations (you can turn around, do a head stand, etc.).
- Supports most humanoid avatars which are correctly imported as humanoid via Unity Mecanim.

The quickest way to experiment with puppeteering avatars is by previewing the Pose Avatar example scene. You can control Unity-chan in this demo scene and examine how it is setup or see further below for instructions on how to add your own avatar.

Notes:

- You can enable/disable the Visuals child of a puppet to visualize the puppet.
- At the moment ensure that the 3d model in the view starts aligned with the global world axes (temporary constraint). After loading, you can move the model however you'd like.
- Unity-chan is made available under the following license: https://unity-chan.com/contents/license_en/

Adding a New Avatar

1. Import the humanoid character into Unity.
2. Under the Rig import settings, ensure it is set to Humanoid and that the Avatar Definition is created.
3. Verify under Configure that there are no errors or warnings detected.
4. When in the Avatar Configuration window, press the Enforce T-pose option in the right window, it is recommended to copy the model in this window for pasting into your game scene since it is enforced in a good T-Pose (makes hooking up to the Puppet easier).
5. Paste the model into the scene you want the avatar in.
6. Add an Avatar component on the pasted model.
7. Assign the Pose Joint Provider to the Puppet in the scene and assign the Animator to the model's animator.
8. Play! That's it!





© 2024-2025 G8gaming Ltd.

Avatar 3D Model Recommendations

The Avatar script can support a wide range of humanoids out-of-the-box. However, there are some guidelines to ensure quality tracking.

- Make sure the core bones are assigned in the Avatar configuration window: Hips, Spine, Chest, Neck, Head, Shoulders, Upper Arms, Lower Arms, Hands, Upper Legs, Lower Legs, Feet.
- The hips should ideally be the root of all the core bones. See Unity-chan's parenting structure. If by chance your model has a different parenting structure, it is usually straight forward to adjust this in software such as Blender as necessary. This is not necessary, but puppeteering quality may suffer otherwise.

Creating Avatars at Runtime

Creating avatars at runtime is supported. It is strongly recommended to use a prefab based approach rather than adding components at runtime.

1. Configure your 3D model correctly and add the Avatar script.
2. Assign all the required fields and test that the avatar is working as expected in the example scene.
3. Create a prefab out of this avatar.

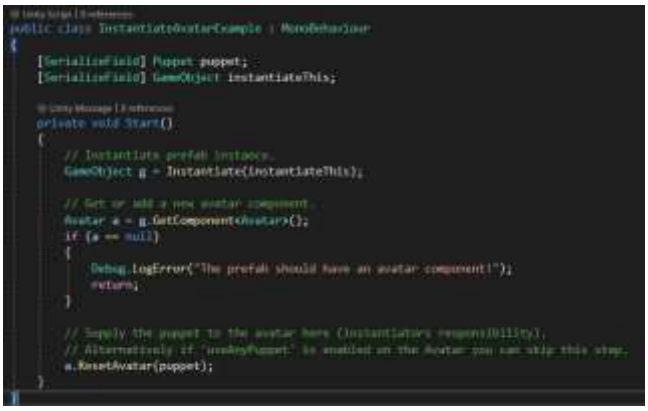
When you want to instantiate a new instance of this prefab the primary issue is that the Puppet may not be a part of the prefab (so the avatar would spawn with a missing provider). The following 2 approaches are recommended in this case:

Option A:

- The puppet must be in the scene by default.
- All Avatars spawned into the scene must have 'Use Any Puppet' enabled. Avatars will arbitrarily bind onto the puppet found in the scene (not recommended in advanced use cases).

Option B:

1. Instantiate the prefab in world space without any transformations.
2. Get the Avatar component and call Avatar.Reset(), supply the joint provider/puppet as an argument.
3. The avatar should be functioning correctly.



© 2024-2025 G8gaming Ltd.

Avatar Controller Examples

After being instantiated, Avatars can be arbitrarily moved and rotated. Tracking4All provides 3 example Avatar controllers in the Scripts/Examples folder which may be of interest depending on your use case. These example avatar controllers are excellent starting points for more advanced specific use cases involving avatar movement:

Example2DAvatarController: allows avatars to be moved in 2D (up/down, left/right) according to normalized landmark coordinates weighted by arbitrary scalers.

HeadBodyIKControllerExample: for use with standalone face tracking, allows the character's spine to move to follow the head position.
FollowAvatarControllerExample: allows avatars to approximately follow/overlay onto Webcam/video feed. Due to the wide variety of potential proportions for avatars, includes configuration options for pivot and avatar rescaling. Please use the segmentation scenes as a baseline. You can disable the segmentation for performance reasons if desired.

- Pivot Bones: The Avatar is translated to the average of the corresponding landmarks (i.e. this is like the 'pin' that the avatar will stick to).
- Scale Pair Bones: The Avatar is 'scaled' to the pairwise differences between the corresponding landmarks in this collection subject to the visibility cutoff value.
- Visibility Cutoff: Scale pair bones that do not reach this visibility threshold are ignored (helps to reduce noise when tracking confidence is low for some bones).
- Corrective Scale Factor: Literally just scale the avatar about the pivot (useful for strange proportion characters).
- Push Up: Translate the avatar up or down (again, useful for strangely proportioned characters).

© 2024-2025 G8gaming Ltd.

Human Segmentation

Tracking4All can be used to coarsely identify what parts of the image are a human versus the background environment (segmentation). This information can be used to mask the player from the webcam image, mask the environment, etc. Samples are included in the example scenes labeled with segmentation.

The segmentation mask is represented by an array of floats that range from 0 to 1. Each segmentation value corresponds to a pixel in the video image where:

- A value that is closer to 0 means the pixel is likely the background environment (not the human being tracked).
- A value that is closer to 1 means the pixel is likely the human being tracked.



Segmentation Shader Examples

The SegmentationVisualizer2D shows an example of using the segmentation mask data in combination with a shader to replace a tracked person with a solid color. There are two exemplar shaders available in the Tracking4All/Shaders/Segmentation folder. Both of these shaders are general-purpose and not particularly designed for performance:

- SegmentationCutoutUI: replaces a value greater than a threshold with solid color (as seen in above image).
- SegmentationInverseCutoutUI: replaces a value less than a threshold with solid color (the background would be removed leaving only the human).

Please note that the primary way to interact with and modify segmentation data is using the GPU (buffers and shaders). While it is relatively straightforward to read the segmentation data on the CPU this is strongly discouraged where possible for performance reasons.

Temporary Additional Notes: Currently it is possible for the segmentation mask to be (usually slightly) out of sync from the frame it belongs to (this is because segmentation data must be asynchronously copied to the GPU before being displayed). If you notice a substantial mismatch between the underlying frame and segmentation mask you can adjust the Artificial Screen Latency parameter in MPUPoseSolution/MPUHolisticPHSolution to add a little latency to sync them better. In the future this synchronization will happen automatically.

© 2024-2025 G8gaming Ltd.

Holistic Hand-Pose (Holistic Lite)

It is possible to track both hands and pose simultaneously using the experimental Holistic Lite solution.

See the scenes in the Examples folder for how this feature can be used. There is a dedicated Holistic Lite Puppet which combines both the pose and hand puppets together. You can choose to puppeteer any combination of; pose, right hand, and left hand through Avatars.



Note: In contrast to other solutions. There is heavy post processing on the raw holistic lite tracking data! This means the Holistic Lite Puppet may not exactly reflect the raw tracking data at all times (usually to good effect).

Holistic Lite Configuration Options

Presently, the most useful parameter to change is probably the Minimum Pose Blend in the Holistic Hands Modifiers (note there are 2 – one for each hand!)

- A high value will cause the system to prefer the orientation of the pose solution more then the true orientation of the hands which tends to produce visually better results in questionable environments.
- In contrast, a low value means the system will infer orientation from the hand data more. The range of 0.4 in good environments and 0.7+ in suboptimal environments is recommended.

Using the EuroSmooth modifier is recommended on the underlying Pose Solution since this tends to reduce noise on the base pose that the hands are infer orientation from. In controlled environments, you may swap this to SimpleSmooth for more responsiveness.

Holistic Lite Limitations

It is important to understand that there are currently limitations to this solution when targeting a broad audience with generic cameras. The primary issue you may face when using this solution is that the camera and environmental conditions will have a considerable impact on tracking quality.

© 2024-2025 G8gaming Ltd.

When moving quickly, the images of the hands on the camera can smear and blur to the point that they are unrecognizable. This problem worsens with; poor lighting, a bad camera, and fast movements. In these cases, most detectors cannot be expected to perform well.

Tracking4All attempts to circumvent these issues by filtering out obviously bad poses and trying to 'guess' frames when hand tracking is intermittently lost. These systems will improve with time.

Additional known hand issues mostly relating to the default mediapipe data provider (solutions to which are being researched):

- Certain unusual hand poses can give poor landmark data (due to a Mediapipe bug).
- Moving very fast can cause hand popping (due to lose of hand tracking then needing to 'catch up' to a big change).
- Bone between wrist and forearm collapses for very low holistic blend parameters (technically unrelated to tracking and instead related to how rotations are blended on 3D models).

In general, it is recommended to prefer the standalone Pose Solution if you can for most stable use cases.

Avatars with Face Tracking

Tracking4All supports animating the face of avatars through the standard 52 blendshapes/shapekeys approach. There are a variety of ways to create characters that also support face tracking.

Consider the sample character included in the FaceTrackingExample scene. You can find the blendshapes that this character uses under the Face child gameobject -> Skinned Mesh Renderer -> BlendShapes dropdown. It is a good idea to familiarize yourself with what blendshapes are and how they work by inspecting them and playing with the sliders in the dropdown.





© 2024-2025 G8gaming Ltd.

Vroid Character Pipeline for Face Tracking

The example character was created using the free software Vroid Studio. After creating the character, the blendshapes for face tracking were automatically added using the free NEB Plugin (see the following comprehensive guide the author for this plugin has posted): https://youtu.be/ZQh7BQ_qQYE?si=uEqpykhLOW349Ror).

Disclaimer: G8gaming Ltd has no affiliation with Vroid Studio or the creator of the NEB Plugin or associated video. All rights reserved to their respective owners.

Note: Any character model is supported as long as it has the standard 52 blendshapes. You need not necessarily use the above-mentioned pipeline. Any character that has the blendshapes is supported.

© 2024-2025 G8gaming Ltd.

Advanced Usage

Tracking4All supports the addition of customized backends/data providers. This and immediately subsequent sections will be useful to read for background on how to add customized backends and extend the framework core.

In advanced use cases you can inherit from the classes used to power Tracking4All and implement custom functionality and additional data you want to support while also remaining compatible with most Tracking4All features.

You can also reference the source code explicitly to build custom Solutions, Adapters, and Detection layers.

In general, the framework is designed so you use high-level components, sourcing data from Providers is the most stable aspect of the framework.

Providers

Central to Tracking4All is the idea of providing data. This is embodied in the IProvider interface definition. Most systems and layers in the framework interact with one another through these providers.

Provider Lifecycle

Every IProvider must include a reference to an IProviderLifecycle. The lifecycle aspect of a provider provides information such as provider readiness as well as some very useful events which are supported by most solutions providing data in the framework:

- OnReady: the provider is set up and available to be read from.
    - You are guaranteed to receive this message when subscribing to a provider before receiving any other message. It is therefore usable for setting up any dependent resources/allocating memory.
- OnDisposed: when the provider is no longer able to provide data.
    - You will usually receive this message before a provider disables itself.
    - If you're looking to dispose of any allocated memory it can be done here, but it is also strongly recommended to dispose of memory independent to a provider to ensure cleanup (ex: dispose here + OnDestroy or OnDisable).
- OnUpdated: when the provider has new data available to be read.
- OnTimedOut: when the provider has not been providing data for some time (by default this timeout time is set in the Tracking4All scene script).
- OnResumed: when the provider has just started providing data again after being timed out.

© 2024-2025 G8gaming Ltd.

Adapter Paradigm and Standardized Adapters

Building on the idea of Providers are Adapters. Adapters are "Providers with Lifecycle" that additionally 'transform data'. Adapters are the most common form of concrete implementation of Providers. They encapsulate the common need in the framework to process data and provide transformed data. 'Transformating data' can mean converting from an arbitrary backend data format to standardized data types, but it can also mean modifying lifecycle to be Unity main thread safe for example.

Adapters all inherit from AdapterBase. This base class holds the skeleton of the more standardized and specialized adapters. You can think of the AdapterBase as holding shared internals which the standardized adapters 'control' to perform their functions.

There are several standardized adapters which will be briefly outlined here by use case:

> BackendAdapter: the 'entry point adapter' offers the highest degree of control and is meant to be hooked into backends. Converts from any arbitrary data type to an IProvider with lifecycle. Requires: calling Ready, Update, Dispose according to your backend.
>
> ProxyAdapter: the 'chaining adapter' takes as input an IProvider and converts to another datatype. Lifecycle wise will simply propagate the lifecycle of the input IProvider.
>
> UnityAdapter: the 'endpoint adapter' is a handy adapter which takes as input an IProvider and hooks onto the lifecycle of that IProvider to convert it to be (unity) main thread safe. Most front-facing data accessible to you will run through this type of adapter at least once to make it unity main thread safe from the backend. Requires: calling Ready, Tick, Dispose.
>
> SolutionAdapter: a variant of UnityAdapter that includes IModifier support to modify data generically (reduces redundancy by allowing Modifiers to implement common operations such as smoothing rather than coupling common implementation into multiple adapters).

At a high level, the idea is to daisychain these adapters from layer to layer forming isolated layers from backend to standardized Tracking4All data structures.

IProvider and InterfaceProvider Peculiarities

This applies if you are using Providers and InterfaceProviders in your code (ex: LandmarkProvider, JointProvider, etc.) AND are doing checks to see if interfaces are null. If you want to avoid these peculiarities, just use concrete classes instead.

The IProvider of a serialized interface provider (ex: LandmarkProvider) will NEVER evaluate as null.

- You can **ONLY** reliably check if an interface provider has an underlying null interface by checking InterfaceProvider.HasInterface or InterfaceProvider.Null.
- Be especially careful when implicitly casting an InterfaceProvider to an interface and then checking if that interface is null (it never will be!)

Solution: if you want to treat an IProvider derived from a serialized InterfaceProvider as nullable use the Helpers.Nullable() extension method, it will return null or the IProvider correctly. Feel free to contact if there is any confusion.

© 2024-2025 G8gaming Ltd.

Optimization Tips

- Adjusting the model complexity will substantially effect performance.
- Using Holistic vs standalone solutions will have a massive impact on frame rates. Use the simplest solution and settings you can get away with.
- You can disable the Visualizer components which display the solution tracking output, data will still be updated as expected.
- You can disable the visuals on Puppets to further improve performance.
- On mobile platforms, shaders and materials will have a massive hit on framerate. Swap out materials and shaders to lower quality versions to preserve performance on mobile platforms.
- It is recommended to reduce camera resolution in production environments and mobile platforms. You do not need a very high-resolution camera to get good results from most detectors (and lower resolution will improve performance drastically).
- Storage space requirements can be improved by deleting unneeded .bytes files in the /Assets/StreamingAssets folder and /Assets/Tracking4All/Third Party/MediaPipeUnity/Core/PackageResources/MediaPipe/ folders (heavy variants or unused solutions can be deleted if not used).

Support

Getting Support

Write to support@g8gaming.ca

Consider joining the Discord: https://discord.gg/7qHpEg8vPH

License and Acknowledgements

License Information

You should have gotten a LICENSE file in your assets folder.

Otherwise see: https://www.tracking4all.com/eula

Credits and Acknowledgements

ganthefans! Discord community for their advice and suggestions.

- Particularly special thanks to:
- phoenix: for detailed feedback and networked webcam help
- Dral: for detailed UX feedback and many bug reports

Homuler MediaPipe Unity Plugin: is used to provide the embedded Unity support.

© 2024-2025 G8gaming Ltd.